

Documentación del Preproyecto

Construcción de un Compilador y Generación de AST

Índice

1. Idea general	2
2. Primera etapa: Flex	2
3. Segunda etapa: Bison	3
4. Tercera etapa: AST	3
5. ¿Qué sabemos después del AST?	4
6. Cuarta etapa: análisis semántico	4
7. La misma entrada a través de todas las etapas	5
8. ¿Por qué necesitamos el AST?	6
9. Integración de las etapas	7
10. Conclusión	7

1. Idea general

La construcción del compilador puede entenderse como una transformación progresiva del programa.

Cada etapa agrega información sobre el código anterior:

Flex identifica → **Bison estructura** → **AST representa** → **Semántica interpreta**

Para entender el proceso, utilizaremos el mismo ejemplo durante todas las etapas:

```
x = 10
```

La misma entrada va adquiriendo significado progresivamente.

2. Primera etapa: Flex

Flex se encarga del análisis léxico. Su tarea es recorrer el texto y reconocer los elementos que pertenecen al lenguaje.

Para cada elemento identifica dos cosas:

- su **tipo de token**;
- su **valor**.

Tomemos:

```
x = 10
```

Flex reconoce tres elementos:

```
ID("x")  
ASSIGN  
NUMBER(10)
```

En esta etapa sabemos que:

- **x** es un identificador;
- **=** es un operador de asignación;
- **10** es un número.

Por lo tanto, Flex nos permite pasar de texto a una secuencia de tokens con sus respectivos valores:

$$x = 10 \longrightarrow ID("x") \quad ASSIGN \quad NUMBER(10)$$

Sin embargo, todavía no sabemos qué relación existe entre estos elementos.

Sabemos que existe un identificador llamado **x** y un número cuyo valor es **10**, pero todavía no sabemos que estamos frente a una asignación.

3. Segunda etapa: Bison

Bison recibe los tokens generados por Flex y utiliza la gramática para determinar cómo se relacionan entre sí.

Recibimos:

```
ID("x") ASSIGN NUMBER(10)
```

y la gramática puede indicar conceptualmente:

```
Asignacion := ID ASSIGN Expresion
Expresion  := NUMBER
```

Entonces Bison reconoce que la secuencia completa representa una **asignación**.

Ahora ya no tenemos solamente elementos separados:

```
ID("x")
ASSIGN
NUMBER(10)
```

sino una estructura:

```
Asignacion
  ID("x")
  NUMBER(10)
```

Por lo tanto:

Tokens	→	Estructura sintáctica
--------	---	-----------------------

La información que conocemos ahora es mayor.

Ya sabemos que:

x = 10

es una estructura válida de tipo **asignación**.

Pero esta estructura todavía debe convertirse en una representación que podamos utilizar cómodamente en las siguientes etapas.

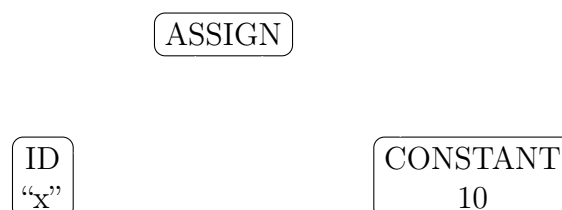
4. Tercera etapa: AST

A partir de la estructura reconocida por Bison podemos construir un Árbol de Sintaxis Abstracta.

Para nuestro ejemplo:

```
x = 10
```

el AST puede representarse como:



Ahora la estructura queda representada mediante nodos.
El nodo **ASSIGN** representa la asignación y contiene dos partes:

- un nodo que representa la variable **x**;
- un nodo que representa el valor **10**.

Podemos expresar el árbol de manera conceptual como:

```
ASSIGN
  ID("x")
  CONSTANT(10)
```

El AST no solamente nos dice qué tokens aparecieron. También conserva las relaciones entre ellos.

Por eso podemos pensar el proceso como:

Texto → Tokens → Estructura → AST

5. ¿Qué sabemos después del AST?

Al llegar al AST ya tenemos una representación estructurada del programa.
Para nuestro ejemplo:

```
ASSIGN
  ID("x")
  CONSTANT(10)
```

podemos identificar claramente:

- **ASSIGN**: estamos frente a una asignación;
- **ID("x")**: la asignación involucra al identificador **x**;
- **CONSTANT(10)**: el valor involucrado es **10**.

Sin embargo, todavía falta una cuestión importante:

¿Qué significa esta estructura?

Ahí comienza el análisis semántico.

6. Cuarta etapa: análisis semántico

El análisis semántico utiliza el AST para determinar el significado de cada estructura dentro del lenguaje.

Partimos de:

```
ASSIGN
  ID("x")
  CONSTANT(10)
```

Semánticamente podemos interpretar:

“Asignar el valor 10 a la variable x.”

Ahora podemos realizar comprobaciones que no corresponden ni a Flex ni a Bison.
Por ejemplo:

- ¿La variable **x** fue declarada?
- ¿Qué tipo tiene **x**?
- ¿El valor 10 es compatible con ese tipo?
- ¿La asignación está permitida por las reglas del lenguaje?

Supongamos que anteriormente se declaró:

```
int x
```

La tabla de símbolos podría contener conceptualmente:

```
x -> int
```

Entonces, al recorrer:

```
ASSIGN
  ID("x")
  CONSTANT(10)
```

podemos comprobar:

$$\text{tipo}(x) = \text{int}$$

y:

$$\text{tipo}(10) = \text{int}$$

Por lo tanto:

La asignación es semánticamente válida

7. La misma entrada a través de todas las etapas

Podemos resumir todo el proceso utilizando nuevamente:

```
x = 10
```

1. Flex

Primero identifica los elementos:

```
ID("x")
ASSIGN
NUMBER(10)
```

Sabemos:

qué elementos hay y qué valor tienen

2. Bison

Luego determina cómo están relacionados:

```
Asignacion
  ID("x")
  NUMBER(10)
```

Sabemos:

qué estructura forman esos elementos

3. AST

La estructura se transforma en un árbol:

```
ASSIGN
  ID("x")
  CONSTANT(10)
```

Tenemos:

una representación estructurada sobre la que podemos trabajar

4. Semántica

Finalmente podemos interpretar:

“Asignar el valor 10 a la variable x.”

y comprobar si esa operación tiene sentido dentro del lenguaje.

8. ¿Por qué necesitamos el AST?

Podríamos preguntarnos por qué no realizar directamente el análisis semántico a partir de los tokens.

El problema es que los tokens solamente describen elementos individuales.

Por ejemplo:

```
ID("x") ASSIGN NUMBER(10)
```

nos dice que existen esos tres elementos, pero no proporciona una estructura conveniente para realizar operaciones sobre ellos.

El AST organiza esa información:

```
ASSIGN
  ID("x")
  CONSTANT(10)
```

De esta manera, las etapas posteriores pueden recorrer el árbol y trabajar directamente con las relaciones entre las partes del programa.

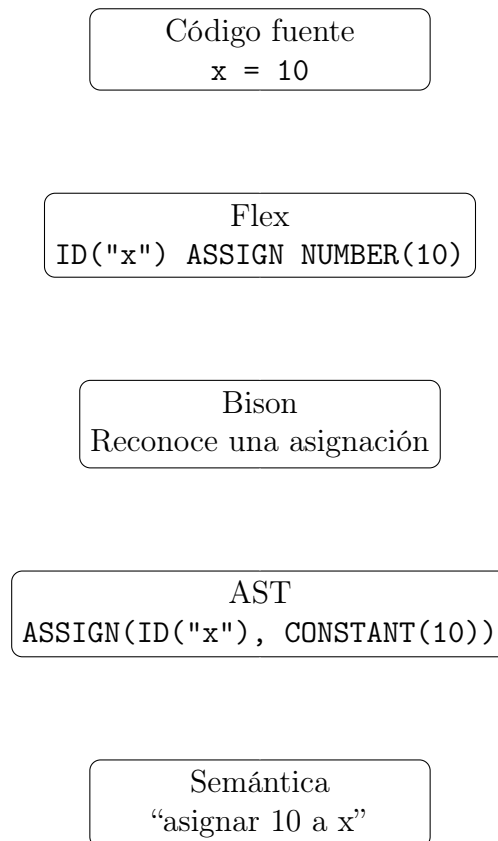
Por ejemplo:

- el análisis semántico puede verificar tipos;

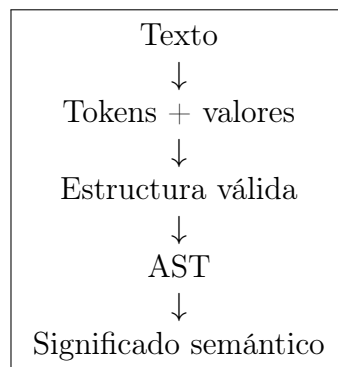
- un intérprete puede evaluar expresiones;
- un generador de código puede traducir las operaciones;
- otras herramientas pueden analizar o transformar el programa.

9. Integración de las etapas

El flujo completo puede verse de la siguiente manera:



Así, cada etapa aporta un nivel de comprensión:



10. Conclusión

La función de cada etapa puede resumirse de manera sencilla:

Flex identifica qué hay y qué valor tiene.

Bison determina cómo se organizan esos elementos.

El **AST** representa esa organización mediante un árbol.

La **semántica** determina qué significa esa estructura.

En nuestro ejemplo, partimos de:

```
x = 10
```

y progresivamente obtenemos:

```
ID("x") ASSIGN NUMBER(10)
```

luego:

```
ASSIGN
  ID("x")
  CONSTANT(10)
```

y finalmente podemos interpretar:

“Asignar el valor 10 a la variable x.”

El **AST** es, por lo tanto, el punto de conexión entre el reconocimiento sintáctico realizado por **Bison** y las etapas posteriores que necesitan comprender y trabajar con el significado del programa.